



FIREBRAND

LYCEUM EDUCATION GROUP

Data Processing and Logic Optimisation

Power Platform Pathway | Session 6

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Data Processing and Logic Optimisation in Power Apps

Lesson Objectives

Apply

- Apply logic to process data effectively

Analyse

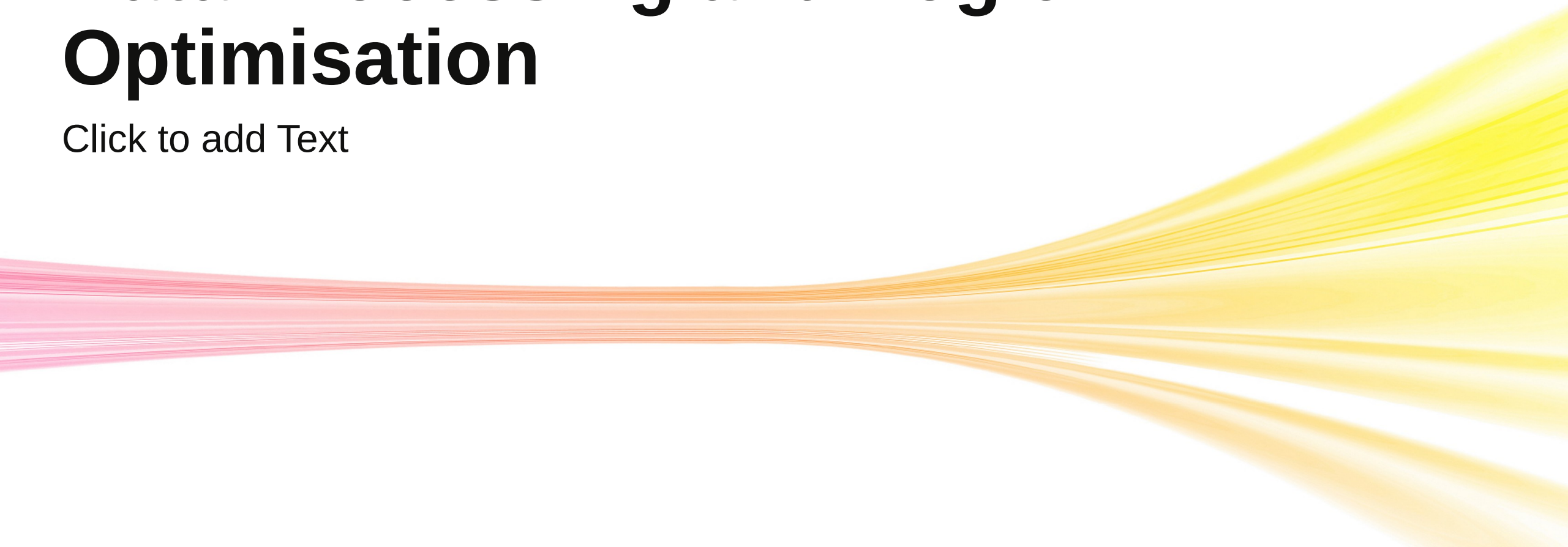
- Analyse inefficiencies in app behaviour

Evaluate

- Evaluate optimisation strategies

Data Processing and Logic Optimisation

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient from light pink on the left, through orange, to bright yellow on the right. They have a soft, ethereal quality, resembling smoke or light trails.

Common Performance Pitfalls

- Repeating the same expensive formula in several places instead of calculating it once
- Rebuilding a whole collection when only one record actually changed

Optimise Formulas with Scoped Logic

- Avoid global variable clutter
- Compute formulas exactly once
- Use local scoped contexts
- Discard values automatically afterward

Production Code Transformation:

// INEFFICIENT: Recalculates identical heavy math twice

```
If(  
    Sum(colOrders, Amount) > 5000,  
    Color.Orange,  
    If(Sum(colOrders, Amount) > 2000, Color.Yellow, Color.Green)  
)
```

// OPTIMISED: Computes calculation exactly ONCE in memory

```
With(  
    { TotalSpent: Sum(colOrders, Amount) },  
    If(  
        TotalSpent > 5000,  
        Color.Orange,  
        If(TotalSpent > 2000, Color.Yellow, Color.Green))  
)
```


Filtering Efficiently

Filter() narrows a table down before displaying it — apply filters as early as possible

Code Example:

```
Filter(Products, Category = "Electronics" && InStock = true)
```

Eradicate Sequential Loading Lag

- Sequential logic blocks interfaces
- Apps wait for queries linearly
- Asynchronous requests run simultaneously
- Reduce loading times drastically

Operational Execution Flow:

// LEGACY SEQUENTIAL FLOW (Blocking Threads)

[App Start] → (Wait) Load Regions → (Wait) Load Status → Render Screen

// MODERN ASYNCHRONOUS FLOW (Parallel Processes)

[App Start] → Concurrent(Load Regions, Load Status) → Render Screen

Concurrent Multi-Thread Queries

Implementing Asynchronous Task Loading

- Wrap non-dependent streams together
- Fetch independent tables simultaneously
- Maximise mobile client efficiency
- Optimise browser thread processing

Concurrent Code

// Place inside App.OnStart or Screen.OnVisible

```
Concurrent(  
    ClearCollect(colRegions, AzureSQL.GetRegions()),  
    ClearCollect(colCategories, AzureSQL.GetCategories()),  
    ClearCollect(colStatus, AzureSQL.GetStatusToggles())  
)
```

Resolving Nested Gallery Loops

- Avoid lookups inside gallery fields
- Repeating code triggers processor drain
- Reference scoped relational pointers
- Clean browser compilation overhead

Structural Grid Optimisation

// INEFFICIENT: Runs lookups for every single visible row

LookUp(Dataverse_Managers, EmployeeID = ThisItem.SupervisorID).FullName

// OPTIMISED: Leverages internal flat database relationships

ThisItem.Supervisor.FullName

Mastering Data Delegation Limits

- Cloud servers filter large records
- Avoid downloading raw tables locally
- Spot blue underline warnings instantly
- Keep apps scaling past limits

Delegation Compatibility Matrix:

Data Source	Delegable Functions	Incompatible Functions (Local Blocks)
Dataverse / SQL	Filter(), Sort(), Lookup()	Search(), Text(), Complex calculations
SharePoint	Basic operators, simple filters	Nested logic, data type conversions

Telemetry Debugging: App Monitor



- Run live data diagnostics
- Log exact network call durations
- Pinpoint hidden UI lag spots
- Inspect HTTP payload responses

3 Step Diagnostics Process:

1. **Launch Trace:** Open the advanced tools panel and initiate the Monitor session
2. **Isolate Operations:** Click target sluggish controls to isolate their code performance
3. **Analyse Payloads:** Review lines displaying high network durations or delegation flags

Spotting Performance Red Flags

UI Flags & Limits

- **The Blue Wavy Underline:** Power Apps places a blue wavy underline beneath expressions that cannot be offloaded to the server
- **The Warning Triangle:** A yellow warning icon appears next to the screen name in the Tree View, signaling a structural optimisation flaw
- **The 500-Row Sandbox Trap:** When a delegation warning is ignored, the app only downloads the first 500 records from the server. The rest of the database becomes invisible to the user

Enterprise Scale Settings

To change the default local processing row limit during testing:

- Navigate to **Settings → General**
- Scroll down to **Data Row Limit**
- Temporarily raise the limit from **500** to **2,000** to test if your search queries drop records

Rules of Asynchronous Processing

The Dependency Trap

- Formulas placed inside a Concurrent() block execute at the exact same time
- Thread B **cannot** read or filter data that is still being fetched or created by Thread A

// WRONG: Thread B crashes because colUser is still empty

```
Concurrent(  
    ClearCollect(colUser, LookUp(Employees, Email = User().Email)),  
    ClearCollect(colOrders, Filter(SalesData, Manager = colUser.Name))  
)
```

// CORRECT: Fetch profile first, then run non-dependent queries

```
ClearCollect(colUser, LookUp(Employees, Email = User().Email));  
Concurrent(  
    ClearCollect(colOrders, Filter(SalesData, Manager = colUser.Name)),  
    ClearCollect(colInventory, AzureSQL.GetWarehouseStock())  
)
```

Monitor Tool Log

Isolating Bottlenecks in Monitor

Critical Operations to Filter:

- **getRows:**

This row marks a database select operation

Focus heavily on this command to measure raw data load times

- **networkDuration:**

Shows the exact time in milliseconds the browser spent waiting for the cloud database to respond

3 Step Root Cause Analysis

- 1. Filter the Feed:** Type `getRows` into the Monitor search bar to clear out generic UI noise
- 2. Open Details:** Click the row showing the highest millisecond duration count
- 3. Inspect Code Line:** Click the **Formula** tab in the right-hand panel to view the exact control and expression causing the delay

Optimisation Challenge



ACTIVITY



Objective: Improve the performance of a provided, poorly performing app

Tasks / Requirements:

- Identify performance bottlenecks within the app
- Propose improvements to formulas, controls and data handling
- Implement and test an optimised version

Key Point: Compare performance before and after your changes

Extension Activity: Document the measurable impact of your optimisation



ACTIVITY



Debugging and Refinement Activity



ACTIVITY



Tasks:

- Trace network call performance logs
- Identify the root cause of performance issues
- Refactor formulas and data operations
- Test and validate improvements
- Document the impact of changes on app performance (e.g. loading speed gains)

Expected Outcome: Improved applications that demonstrate efficient data handling and responsive user experiences



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion



Performance Evaluation Matrix

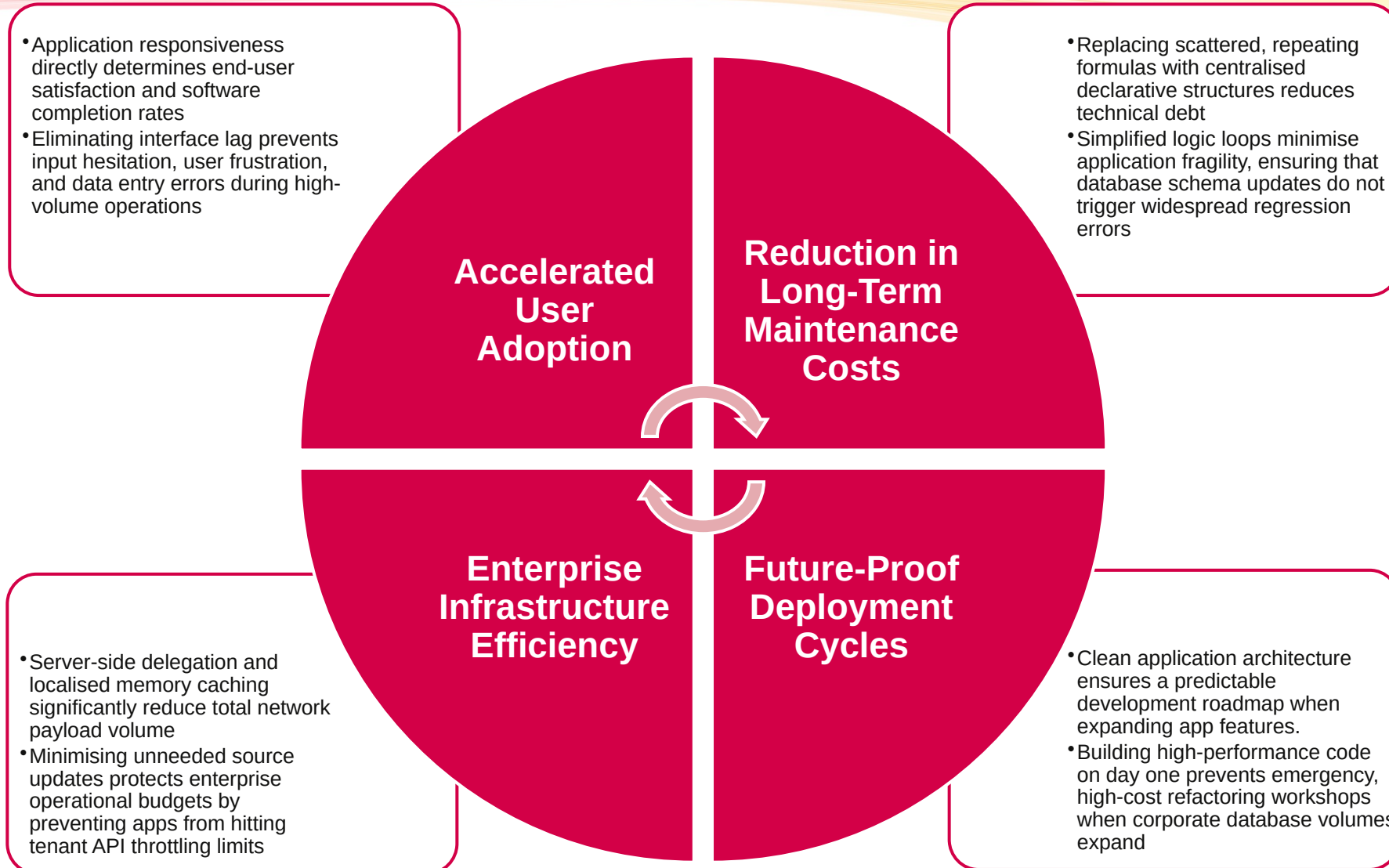
Baseline vs. Optimised

- **Performance Metrics:** Verification of reduced API network call times and the complete elimination of screen-load blocking animations
- **User Experience (UX):** Delivery of fluid interactive button responses, stutter-free gallery scrolling, and instantaneous text search filtering
- **Formula Efficiency:** Implementation of scoped local contexts to minimize memory footprints and reduce global variable sprawl
- **Future Scalability:** Architectural validation confirming the application handles expanding backend data rows without triggering delegation blocks

Structural Performance Comparison

Operational Area	Legacy Monolithic Solution	Refactored Optimised Solution
Boot Sequence	Sequential table loading screens block the UI thread	Asynchronous Concurrent() blocks execute queries simultaneously
Data Processing	Direct lookup queries execute repeatedly inside gallery loops	Relational database paths and flat in-memory collections cache records
Complex Math	Identical heavy expressions evaluate multiple times across forms	Local With() functions calculate values once inside a scoped block
Data Retrieval	Incompatible local data matching hits row limits at 500 records	Delegated server-side filtering scales smoothly to 100,000+ records

The Business Value



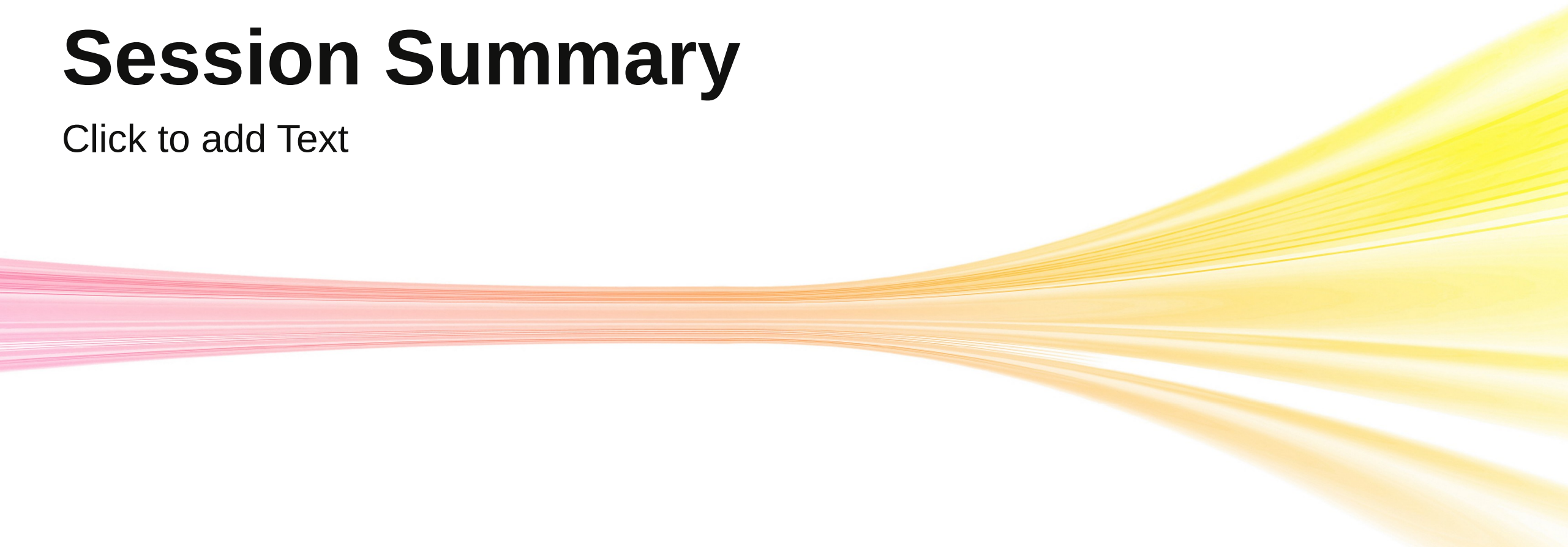
Stretch and Challenge

The Mission: Produce scalable architecture

- **Task 1:** Redesign a section of the application to improve efficiency
- **Task 2:** Reduce unnecessary processing and data operations
- **Task 3:** Apply alternative approaches to formula and data management
- **Task 4:** Justify your structural data caching, collection handling, and server delegation choices based on solution usability

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Searching and Sorting Algorithms in C#

Lesson Objectives

Apply

- Apply logic to process data effectively

Analyse

- Analyse inefficiencies in app behaviour

Evaluate

- Evaluate optimisation strategies

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

